

# Algorithms of `mpc_cuda`

Correctly-rounded multiple precision and a register-resident

fixed-precision fast path for CUDA

tkouya

June 18, 2026

## Abstract

`mpc_cuda` ports the GNU multiple-precision stack—GMP (integers), MPFR (correctly-rounded binary floating point) and MPC (complex)—so that it runs inside CUDA kernels on both the host and the device. This note documents the *algorithms*, in the spirit of MPFR’s `algorithms.tex`, and is meant to be citable from a paper. We separate two layers. The *runtime* layer (`cu_mpfr_*`, `cu_mpc_*`) reproduces the upstream MPFR/MPC algorithms verbatim, with one deliberate algorithmic substitution—a self-contained, `mpz`-based correctly-rounded division that replaces MPFR’s generic divide-and-conquer path, which miscompiles under `nvcc`. The *fixed-precision* layer provides register-resident types `cu_frexp<p>` and `cu_fcomplex<p>` whose precision  $p$  is a compile-time constant; their basic arithmetic is bit-for-bit identical to MPFR round-to-nearest-even (RNDN) and to MPC’s per-component MPC\_RNDNN, while their elementary functions are *faithful* (high accuracy,  $\lesssim 1$  ULP, not correctly rounded). We give the storage model, the rounding kernel, the schoolbook multiply, the exact fused fmms/fmma that makes complex multiply correctly rounded, and the working-precision Newton / argument-reduction schemes of the elementary functions.

## Contents

|          |                                                                       |          |
|----------|-----------------------------------------------------------------------|----------|
| <b>1</b> | <b>Scope and the two layers</b>                                       | <b>1</b> |
| <b>2</b> | <b>The runtime layer: departures from MPFR/MPC</b>                    | <b>2</b> |
| 2.1      | Memory management: the per-thread bump arena . . . . .                | 2        |
| 2.2      | Device-correct division . . . . .                                     | 3        |
| 2.3      | Functions that are host-only . . . . .                                | 4        |
| <b>3</b> | <b>Fixed-precision real type <code>cu_frexp&lt;p&gt;</code></b>       | <b>4</b> |
| 3.1      | Storage . . . . .                                                     | 4        |
| 3.2      | The rounding kernel <code>finalize</code> . . . . .                   | 4        |
| 3.3      | Multiplication . . . . .                                              | 5        |
| 3.4      | Addition and subtraction . . . . .                                    | 5        |
| 3.5      | Cost . . . . .                                                        | 5        |
| <b>4</b> | <b>Fixed-precision complex type <code>cu_fcomplex&lt;p&gt;</code></b> | <b>6</b> |
| 4.1      | Exact fused fmms/fmma . . . . .                                       | 6        |
| 4.2      | Complex multiply . . . . .                                            | 6        |
| <b>5</b> | <b>Fixed-precision elementary functions</b>                           | <b>7</b> |
| 5.1      | Newton kernels (division, square root, cube root) . . . . .           | 7        |
| 5.2      | Transcendental functions: range reduction then series . . . . .       | 7        |
| 5.3      | Complex elementary functions . . . . .                                | 8        |

|   |                         |   |
|---|-------------------------|---|
| 6 | Accuracy and validation | 8 |
| 7 | How to cite             | 9 |

## 1 Scope and the two layers

`mpc_cuda` exposes two interoperable numeric layers; their rounding contracts are summarised in Table 1.

|                | Runtime layer<br><code>cu_mpfr_t</code> , <code>cu_mpc_t</code> | Fixed-precision layer<br><code>cu_freal&lt;p&gt;</code> ,<br><code>cu_fcomplex&lt;p&gt;</code> |
|----------------|-----------------------------------------------------------------|------------------------------------------------------------------------------------------------|
| Precision      | runtime (per object)                                            | compile-time constant $p$                                                                      |
| Storage        | heap / bump arena (limb array)                                  | registers ( $\lceil p/64 \rceil$ limbs)                                                        |
| Rounding modes | all MPFR/MPC modes (argument)                                   | RNDN only (no argument)                                                                        |
| Basic arith.   | upstream MPFR/MPC algorithms                                    | bit-exact with MPFR RNDN / MPC_RNDNN                                                           |
| Division       | <code>mpz</code> -based replacement (§2.2)                      | Newton reciprocal (§5)                                                                         |
| Elem. funcs    | upstream MPFR/MPC (correctly rounded)                           | faithful, $\lesssim 1$ ULP (§5)                                                                |

Table 1: The two numeric layers of `mpc_cuda` and their rounding contracts.

The two layers share the same number model, which we fix now.

**Number model.** Following MPFR, a nonzero finite number of precision  $p$  bits is

$$x = s \cdot m \cdot 2^{e-p}, \quad s \in \{-1, +1\}, \quad 2^{p-1} \leq m < 2^p, \quad m \in \mathbb{Z}, \quad (1)$$

i.e. a sign, a  $p$ -bit integer significand  $m$  whose most significant bit (MSB) is set, and an exponent  $e$ . Zero is a distinct flagged value. The significand is stored in  $N = \lceil p/64 \rceil$  64-bit limbs, *left-justified* (MSB in the top bit of the top limb); the  $SB = 64N - p \in \{0, 32\}$  low bits are held zero. This is exactly MPFR’s in-memory layout, which is why results can be compared bit-for-bit against host MPFR.

The remainder is organised as: the runtime layer and its one algorithmic departure from MPFR (§2); the fixed-precision real type (§3); the fixed-precision complex type and the exact fused multiply that makes it correctly rounded (§4); and the fixed-precision elementary functions (§5). §6 collects the accuracy guarantees and validation; §7 explains how to cite this work.

## 2 The runtime layer: departures from MPFR/MPC

The runtime layer is generated from the upstream GMP/MPFR/MPC sources by a mechanical port (`cu_` prefixing, host/device qualification, and removal of host-only I/O). *Algorithmically it is upstream MPFR/MPC*: addition, subtraction, multiplication, square root, the constants, and all transcendentals use MPFR’s own routines and Ziv correctly-rounded loops, so on the device they return the same bits as host MPFR for every rounding mode. The complex layer is MPC unchanged: real/imaginary parts are rounded independently with a packed rounding mode `MPC_RND( $r_{\Re}$ ,  $r_{\Im}$ )`. Two points are worth recording for a citation: the rounding semantics are identical to upstream, and exactly one arithmetic kernel is replaced. One more device-specific mechanism deserves description, because it is essential to making the runtime layer fast rather than merely correct: the memory allocator (§2.1).

## 2.1 Memory management: the per-thread bump arena

Every multiple-precision operation allocates: an `mpfr_t` needs a buffer for its limbs, and a single `cu_mpfr_div` or `cu_mpc_mul` allocates and frees *dozens* of temporaries as it descends its call chain. On the CPU this is cheap; on the device the `malloc/free` is a single global serialising allocator shared by every thread, and when thousands of threads contend it becomes the dominant cost—in this library, the difference between GPU parity and a  $\sim 40\times$  speedup. This is purely an implementation concern (it does not change any result), but it is the reason the runtime layer is usable at scale, so we record the scheme here.

`mpc_cuda` replaces the device heap with a *per-thread bump arena*: one contiguous `cudaMalloc`'d block partitioned into one fixed-size *slab* per resident thread, indexed by global thread id (Figure 1). The three operations are:

- **Allocate** = add the request size to the thread's offset `mpc_cuda_arena_top[tid]` and return the old position—a handful of instructions, no search, no lock, *no cross-thread contention* (threads never touch each other's slabs).
- **Free** = *nothing* (a no-op).
- **Reset** = set the thread's offset back to zero, reclaiming its whole scratch region at once.

Because free is a no-op, everything allocated between two resets stays live: the arena is not a general heap but a *scratchpad for one unit of work*. A kernel does all the allocation for one work item, then calls `mpc_cuda_arena_reset()` before the next. The same hooks back both mini-GMP and, through `mpfr-gmp.c`, MPFR and MPC, so installing the arena accelerates every layer at once. Two fallbacks keep results correct regardless of sizing: with no arena installed (`mpc_cuda_arena_base = NULL`) allocations go to the device `malloc/free`; and if a work item exceeds its slab, *that* allocation silently spills to `malloc` and is freed normally. Overflow therefore costs performance, never correctness.

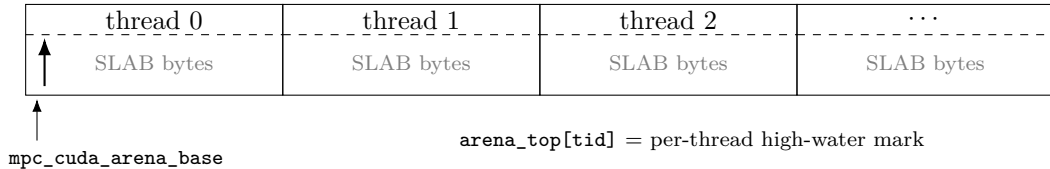


Figure 1: The per-thread bump arena: one `cudaMalloc` block split into a fixed-size slab per resident thread. Allocate bumps the thread's offset; free is a no-op; reset zeroes it. No two threads share a slab, so there is no contention.

**Sizing.** Total arena memory is `LAUNCH × SLAB`: `LAUNCH` the fixed number of resident threads (chosen for occupancy—the deep call chains keep limbs in global-memory slabs, so many warps are needed to hide that latency), and `SLAB` the per-thread scratch, which must hold one work item's peak simultaneously-live allocation. Because free is a no-op, that peak equals everything allocated since the last reset, and it is directly measurable: `mpc_cuda_arena_top[tid]` is the high-water mark, so running one work item without resetting reports the exact byte count. Scratch scales roughly linearly with precision; empirically (at 1024 bits) a real MPFR `axpy/dot` needs  $\approx 32$  KB per thread and a complex MPC one  $\approx 256$  KB (about  $8\times$ , complex chains being deeper). Over-provisioning only costs memory; under-provisioning silently drops onto the slow path.

## 2.2 Device-correct division

MPFR’s generic division (`mpfr_div`, the divide-and-conquer / short-division path of [4, 5]) miscompiles under `nvcc` for precisions of three limbs or more, returning `inf` or garbage on the device. The fault was root-caused to the optimizer but the offending construct resisted isolation, and lower-level building blocks (`mpn_divrem`, mini-GMP `mpz` division, `udiv_qrnd`, `count_leading_zeros`) are all *verified correct* on the device. `mpc_cuda` therefore substitutes, on the device only, a self-contained correctly-rounded division built solely on the trusted `mpz` primitives; the host keeps upstream MPFR. Division is the foundation of the constants and transcendentals (which divide internally), so this substitution is what unblocks the whole runtime layer.

---

**Algorithm 1** `mpfr_div_cuda(u, v, rnd)`: device-correct, correctly-rounded division

---

**Require:** finite nonzero  $u, v$  of precision  $\leq$  that of the destination  $q$  ( $p$  bits); singular cases  $(0, \infty, \text{NaN})$  handled separately as in MPFR.

- 1:  $s \leftarrow \text{sign}(u) \cdot \text{sign}(v)$
  - 2: Build the significands of  $u$  and  $v$  as integers  $M_u, M_v$  (`mpz`)
  - 3:  $\text{shift} \leftarrow p + 2 + (\text{limb count of } v) \cdot 64$  ▷ guard bits for one exact quotient
  - 4:  $\text{num} \leftarrow M_u \ll \text{shift}$
  - 5:  $(Q, R) \leftarrow \text{mpz_tdiv_qr}(\text{num}, M_v)$  ▷ one exact integer division
  - 6: derive the round bit and sticky bit ( $\text{sticky} = [R \neq 0]$  ORed with dropped low bits of  $Q$ )
  - 7: normalise  $Q$  to  $p$  bits, set the quotient exponent from  $e_u - e_v$  and the shift
  - 8: apply the rounding mode  $\text{rnd}$  to  $(Q, \text{round}, \text{sticky})$ ; set sign  $s$
  - 9: **return** `mpfr_check_range(q)` ▷ matches host `mpfr_div` bit-for-bit
- 

The quotient is formed by a single `mpz_tdiv_qr` of the numerator scaled by  $p + 2$  guard bits (plus a full divisor’s worth, so the integer quotient already contains the target significand, a round bit, and enough material to decide sticky). Rounding then follows MPFR exactly, for *all* rounding modes, so the public `cu_mpfr_div` is bit-identical to host `mpfr_div`. The cost is one big-integer division of  $O(p)$  limbs; it is asymptotically the same class as MPFR’s path and is invisible to the caller.

## 2.3 Functions that are host-only

Formatted and string I/O (`mpfr_printf`, `mpfr_get_str` parsing, `mpfr_set_str`, `mpz_out_str`, ...) and the `mpf_t`/`mpq_t` conversions absent from mini-GMP are not available on the device; their device stubs trap if called. These are not algorithmic changes, only surface restrictions, but they bound what a kernel may call.

## 3 Fixed-precision real type `cu_freal<p>`

When the precision is known at compile time, `cu_freal<p>` (header `cu_freal.cuh`) holds the whole number in registers and dispenses with the arena and the runtime-precision dispatch.  $p$  is any positive multiple of 32 (so  $SB \in \{0, 32\}$ ), and the significand occupies  $N = \lceil p/64 \rceil$  limbs. Every operation is bit-exact with MPFR RNDN.

### 3.1 Storage

A value is a triple  $(\text{sign}, \text{exp}, \text{m}[N])$  realising (1):  $\text{sign} \in \{\pm 1\}$ ,  $\text{exp}$  the MPFR exponent, and  $\text{m}$  the left-justified significand (Figure 2). A reserved exponent flags zero. Because the layout matches MPFR’s, conversion to/from a runtime `cu_mpfr_t` of the same precision is a copy.

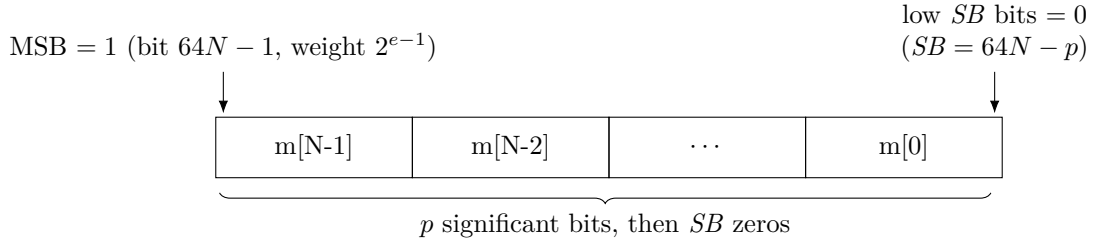


Figure 2: Left-justified significand of `cu_freal<p>`:  $N$  limbs, high to low, MSB always set, low  $SB$  bits forced zero. Identical to MPFR’s in-memory layout.

### 3.2 The rounding kernel `finalize`

All real operations funnel through one routine, `finalize`, which takes an exact magnitude (a little-endian limb buffer  $D$  with a known MSB position and exponent, plus round/sticky material that lies *below*  $D$ ) and produces the RNDN-rounded `cu_freal<p>`. This single point of rounding is what makes the type provably bit-exact with MPFR: every kernel below is responsible only for forming an *exact* intermediate and handing it to `finalize`.

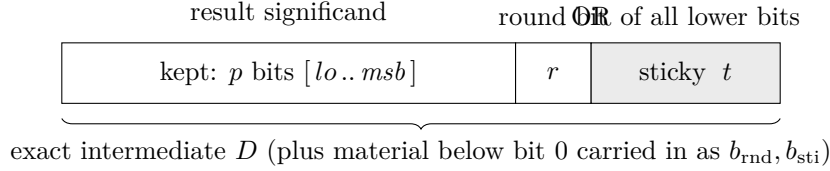


Figure 3: Decision bits used by `finalize`: the  $p$  kept bits, the round bit  $r$  immediately below them, and the sticky bit  $t$  (logical OR of everything still lower). Round-to-nearest-even increments iff  $r \wedge (t \vee \text{lsb})$ .

---

#### Algorithm 2 `finalize`( $D, msb, E_{hi}, b_{rnd}, b_{sti}, s$ )

---

```

1:  $lo \leftarrow msb - p + 1$  ▷ lowest kept bit position in  $D$ 
2: if  $lo > 0$  then
3:    $V \leftarrow \text{bits } [lo..lo + p - 1] \text{ of } D$ ;  $r \leftarrow D_{lo-1}$ ;  $t \leftarrow b_{rnd} \vee b_{sti} \vee (\bigvee_{i < lo-1} D_i)$ 
4: else if  $lo = 0$  then
5:    $V \leftarrow D$ ;  $r \leftarrow b_{rnd}$ ;  $t \leftarrow b_{sti}$ 
6: else
7:    $V \leftarrow D \ll (-lo)$ ;  $r \leftarrow 0$ ;  $t \leftarrow 0$  ▷  $< p$  significant bits: exact
8: end if
9: if  $r \wedge (t \vee (V \bmod 2))$  then ▷ round to nearest, ties to even
10:   $V \leftarrow V + 1$ 
11:  if  $V$  carried out of bit  $p - 1$  then
12:     $V \leftarrow 2^{p-1}$ ;  $E_{hi} \leftarrow E_{hi} + 1$ 
13:  end if
14: end if
15:  $m \leftarrow V \ll SB$ ;  $exp \leftarrow E_{hi} + 1$ ;  $sign \leftarrow s$ 
16: return ( $sign, exp, m$ )
```

---

The three branches handle the normal case, the aligned case, and the fewer-than- $p$ -bit (hence exact) case; the carry-out branch handles the  $1.\bar{1} \rightarrow 10.0$  overflow by bumping the exponent. Conversions from `double` (`from_double`) and to `double` (`to_double`) use the same round-bit/sticky-bit logic on the 53-bit IEEE mantissa.

### 3.3 Multiplication

Multiplication forms the *exact*  $2N$ -limb product with a register-resident schoolbook  $O(N^2)$  multiply (Figure 4), then normalises and rounds it through **finalize**. Because  $p$  is a compile-time constant the double loop is fully unrolled and **ptxas** keeps every limb in a register; on the device each  $64 \times 64 \rightarrow 128$  step is a **mad.lo.cc.u64**/**madc.hi.u64** pair, so the inner product runs with no global-memory traffic.

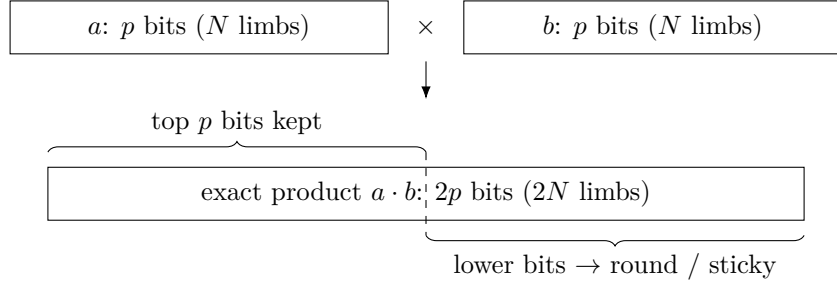


Figure 4: Fixed-precision multiply: the exact  $2p$ -bit product is computed in registers, then its top  $p$  bits are kept and the remainder feeds the round/sticky decision of **finalize**. The result equals  $\circ(a \cdot b)$ , i.e. **mpfr\_mul** at  $p$  bits.

### 3.4 Addition and subtraction

Addition aligns the smaller operand by the exponent difference  $d = e_a - e_b$ , capturing the bits shifted past limb 0 as round/sticky, sums into an  $N+1$ -limb buffer, and rounds (Figure 5). Subtraction of like signs uses a buffer with two guard limbs (128 extra bits) so that catastrophic cancellation still leaves a correctly-rounded result: the exact difference is formed, the leading nonzero bit located, and **finalize** rounds from there. Same-sign/opposite-sign dispatch and the magnitude comparison mirror MPFR's **mpfr\_add**/**mpfr\_sub**.

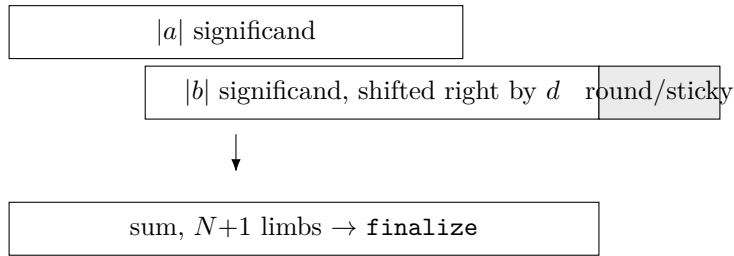


Figure 5: Fixed-precision add: align  $|b|$  by  $d = e_a - e_b$ , keep the shifted-out bits as round/sticky, add, then round once. Subtraction uses two extra guard limbs to absorb cancellation.

### 3.5 Cost

With  $N = \lceil p/64 \rceil$ , multiplication is  $O(N^2)$  word multiplies and add/sub are  $O(N)$ , all register-resident with no allocation. The schoolbook multiply is asymptotically inferior to GMP/MPFR's sub-quadratic multiplication, so the fixed-precision advantage is greatest at low and medium precision (where keeping limbs in registers dominates) and erodes once the working set spills out of registers at high  $p$ .

## 4 Fixed-precision complex type `cu_fcomplex<p>`

`cu_fcomplex<p>` (header `cu_fcomplex.cuh`) is a pair of `cu_frexp<p>` components. Addition and subtraction are componentwise `cu_fadd/cu_fsub`, hence trivially `MPC_RNDNN`. The interesting operation is multiplication, which is made *correctly rounded per component*—bit-exact with MPC’s `MPC_RNDNN`—by avoiding double rounding.

### 4.1 Exact fused `fmms/fmma`

The naive complex product  $\Re = ar br - ai bi$ ,  $\Im = ar bi + ai br$  rounds each of the four real products before subtracting/adding, incurring two roundings per component and so failing to match MPC. Instead `mpc_cuda` evaluates each component with a single rounding via the fused primitives

$$\text{fmms}(a, b, c, d) = \circ(ab - cd), \quad \text{fmma}(a, b, c, d) = \circ(ab + cd), \quad (2)$$

each computed from the two *exact*  $2N$ -limb products combined with a *single* RNDN rounding (Figure 6). Concretely each product  $ab$  and  $cd$  is formed exactly by the schoolbook multiply of §3.3 and normalised to a signed wide term (`sign, exp, M[2N]`); the two wide terms are then aligned by their exponent difference (with three guard limbs), added or subtracted exactly, and rounded once by `finalize`. The result therefore equals  $\circ(ab \pm cd)$  with no intermediate rounding, which is precisely what MPFR’s `mpfr_fmms/mpfr_fmms` (and hence MPC) compute.

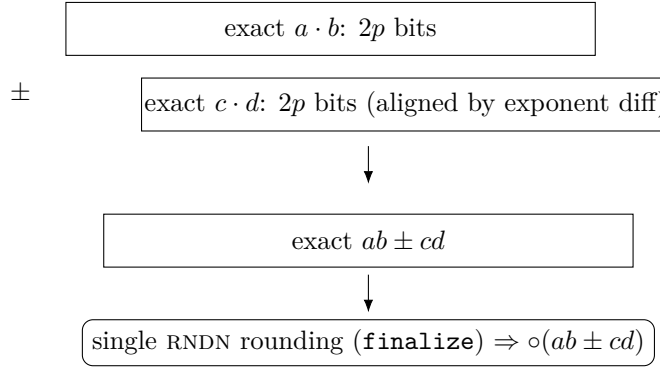


Figure 6: Exact fused `fmms / fmms`: both  $2p$ -bit products are kept exact, aligned, combined, and rounded *once*. No double rounding, so each component matches MPC’s `MPC_RNDNN` bit-for-bit.

### 4.2 Complex multiply

With the fused primitives, complex multiplication is the textbook identity evaluated component-exactly:

$$(ar + i ai)(br + i bi) = \underbrace{\text{fmms}(ar, br, ai, bi)}_{\Re = \circ(ar br - ai bi)} + i \underbrace{\text{fmma}(ar, bi, ai, br)}_{\Im = \circ(ar bi + ai br)}. \quad (3)$$

This is the “ $3M$ -free”, accuracy-first formulation: four exact products, two single-rounded combinations, validated bit-exact against MPC over  $p = 32 \dots 2048$ .

## 5 Fixed-precision elementary functions

The elementary functions (headers `cu_fmath.cuh` for real, `cu_fcmath.cuh` for complex) trade correct rounding for register residency. Each function evaluates at a *working precision*

$$W = p + g, \quad g = \text{CU_FGUARD} = 128 \text{ bits (default)}, \quad (4)$$

using Newton iteration and argument-reduced series in `cu_freal<W>`, then re-rounds the  $W$ -bit result back to  $p$ . Results are *faithful* to about  $p$  bits and agree with MPFR/MPC to  $\lesssim 1$  ULP in the common range. They are deliberately *not* correctly rounded: the table-maker’s dilemma would require an unbounded Ziv loop, incompatible with a fixed register budget. Use the runtime `cu_mpf`/`cu_mpc` API when last-bit correctness is required.

### 5.1 Newton kernels (division, square root, cube root)

Each algebraic function iterates a self-correcting Newton map seeded from the `double` approximation; the number of iterations is  $\lceil \log_2(W/52) \rceil + 1$ , doubling the correct digits each step (Table 2).

| Function             | Quantity                     | Newton update                         | Reconstruction              |
|----------------------|------------------------------|---------------------------------------|-----------------------------|
| <code>cu_fdiv</code> | reciprocal $z \approx 1/m$   | $z \leftarrow z(2 - mz)$              | $a/b = a \cdot (1/b)$       |
| <code>cu_sqrt</code> | rsqrt $y \approx 1/\sqrt{m}$ | $y \leftarrow \frac{1}{2}y(3 - my^2)$ | $\sqrt{m} = m \cdot y$      |
| <code>cu_cbrt</code> | rcbrt $y \approx m^{-1/3}$   | $y \leftarrow y(4 - my^3)/3$          | $\sqrt[3]{m} = m \cdot y^2$ |

Table 2: Division-free Newton iterations used by the fixed-precision algebraic functions.  $m$  is the mantissa after binary scaling into a fixed range; the power of two is reattached afterward.

The reciprocal iteration uses only multiplies and a subtract (no division), which is why even `cu_fdiv` avoids a hardware divide; `cu_div_ui` (division by a small  $< 2^{32}$  integer) is a separate, correctly-rounded limb-by-limb routine used inside the series.

### 5.2 Transcendental functions: range reduction then series

The transcendentals reduce the argument into a small interval, sum a Taylor (or `atanh/atan`) series until the term drops below the working-precision threshold, and reconstruct. The schemes are summarised in Table 3; constants  $\ln 2$  and  $\pi$  are themselves computed at working precision by  $2 \operatorname{atanh}(1/3)$  and Machin’s  $16 \arctan(1/5) - 4 \arctan(1/239)$ .

| Function                  | Reduction                                                                                           | Series / reconstruction                                                         |
|---------------------------|-----------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------|
| <code>cu_exp</code>       | $x = k \ln 2 + r$ , $k = \lfloor x / \ln 2 \rfloor$ ; halve $r \rightarrow r/2^m$ until $ r $ small | $\exp(r) = \sum r^n/n!$ , square $m$ times, scale by $2^k$                      |
| <code>cu_log</code>       | $x = 2^e m$ , $m \in [\frac{1}{\sqrt{2}}, \sqrt{2})$ ; $t = \frac{m-1}{m+1}$                        | $\log m = 2 \operatorname{atanh}(t)$ , add $e \ln 2$                            |
| <code>cu_sin/cos</code>   | $x = k \frac{\pi}{2} + r$ , $ r  \leq \frac{\pi}{4}$ ; halve $r$                                    | Taylor for $\sin r, \cos r$ , double-angle back, quadrant select on $k \bmod 4$ |
| <code>cu_atan</code>      | reduce to $ x  \leq 1$ via $1/x$ ; $a \leftarrow a/(1 + \sqrt{1 + a^2})$                            | $\arctan$ series, scale by $2^{(\#\text{reductions})}$                          |
| <code>cu_asin/acos</code> | $\arcsin x = \arctan(x/\sqrt{1-x^2})$ ; $\arccos = \pi/2 - \arcsin$                                 | via <code>cu_atan</code>                                                        |
| <code>cu_sinh/cosh</code> | via $e^x$ (small- $x$ Taylor for $\sinh$ )                                                          | $\frac{1}{2}(e^x \mp e^{-x})$                                                   |
| <code>cu_atanh</code>     | $ x  \leq \frac{1}{2}$ series; else $\frac{1}{2} \log \frac{1+x}{1-x}$                              | $\operatorname{atanh}$ series / <code>cu_log</code>                             |
| <code>cu_pow</code>       | $x^y = \exp(y \log x)$ ( $x > 0$ ); integer- $y$ branch for $x < 0$                                 | via <code>cu_exp</code> , <code>cu_log</code>                                   |

Table 3: Argument reduction and reconstruction for the fixed-precision transcendentals. All intermediates are `cu_freal<W>` with  $W = p + 128$ .

### 5.3 Complex elementary functions

The complex layer (`cu_fcmath.cuh`: `cu_cexp`, `cu_clog`, `cu_csqrt`, `cu_csin`, `cu_ccos`, `cu_ctan`, `cu_csinh`, `cu_ccosh`, `cu_cdiv`, and `atan2`) is built from the real functions plus the *exact* `fmms` / `fmma` of §4.1 for every squared-modulus and cross term, e.g.

$$|z|^2 = \text{fmma}(\Re z, \Re z, \Im z, \Im z), \quad \log z = \frac{1}{2} \log |z|^2 + i \operatorname{atan2}(\Im z, \Re z), \quad (5)$$

$$e^z = e^{\Re z} (\cos \Im z + i \sin \Im z), \quad \frac{a}{b} = \frac{\text{fmma}(ar, br, ai, bi) + i \text{fmms}(ai, br, ar, bi)}{\text{fmma}(br, br, bi, bi)}. \quad (6)$$

Using the exact fused products for  $|b|^2$  and the numerators keeps the complex divide and the modulus from losing accuracy to double rounding. Like their real counterparts these are faithful,  $\lesssim 1$  ULP vs MPC.

## 6 Accuracy and validation

| Operation                                                             | Guarantee                                           | Validation range      |
|-----------------------------------------------------------------------|-----------------------------------------------------|-----------------------|
| runtime <code>cu_mpfr/cu_mpc</code> (incl. <code>cu_mpfr_div</code> ) | correctly rounded, all modes; bit-identical to host | matches host MPFR/MPC |
| <code>cu_freal</code> $+$ , $-$ , $\times$                            | bit-exact MPFR <code>RNDN</code>                    | $p = 32 \dots 2048$   |
| <code>cu_fcomplex</code> $+$ , $-$ , $\times$                         | bit-exact MPC <code>RNDNN</code>                    | $p = 32 \dots 2048$   |
| <code>cu_fmath/cu_fcmath</code>                                       | faithful, $\lesssim 1$ ULP (not correctly rounded)  | $p = 64 \dots 1024$   |

Table 4: Accuracy contract per operation and the validated precision ranges.

The two basic-arithmetic claims are checked bit-for-bit against the system MPFR and MPC; the elementary-function claim is checked in ULP against MPFR/MPC (the shipped suite reports 0 ULP in its sampled inputs, with  $\leq 1$  ULP the designed bound). The contracts are summarised in Table 4.

## 7 How to cite

This document describes the algorithms of `mpc_cuda`. The two results most likely to be cited are (i) the register-resident fixed-precision arithmetic that is bit-exact with MPFR `RNDN` and MPC `MPC_RNDNN` via a single rounding point and exact fused `fmms` / `fmma` (§3–4), and (ii) the `mpz`-based correctly-rounded division that replaces MPFR’s generic path on the device (§2.2). The companion technical report (`mpc_cuda_report`) gives the performance evaluation, including the GPU/CPU crossover and the  $\sim 140\times$  speedup of `cu_freal` over the runtime GPU path at 1024 bits for AXPY. The runtime layer’s correctly-rounded semantics follow MPFR [2] and MPC [3], themselves built on GMP [1]; the underlying methods are standard [6, 7].

## References

- [1] T. Granlund and the GMP development team, *GNU MP: The GNU Multiple Precision Arithmetic Library*. <https://gmplib.org/>.
- [2] L. Fousse, G. Hanrot, V. Lefèvre, P. Pélicier, P. Zimmermann, “MPFR: A multiple-precision binary floating-point library with correct rounding,” *ACM Trans. Math. Softw.*, 33(2), 2007. <https://www.mpfr.org/>.

- [3] A. Enge, M. Gastineau, P. Théveny, P. Zimmermann, *GNU MPC: A library for multiprecision complex arithmetic with exact rounding*. <https://www.multiprecision.org/mpc/>.
- [4] D. Harvey and P. Zimmermann, “Short Division of Long Integers,” *Proc. 20th IEEE Symp. on Computer Arithmetic (ARITH-20)*, 2011, pp. 7–14.
- [5] N. Möller and T. Granlund, “Improved Division by Invariant Integers,” *IEEE Trans. Computers*, 60(2):165–175, 2011.
- [6] R. P. Brent and P. Zimmermann, *Modern Computer Arithmetic*, Cambridge University Press, 2010.
- [7] J.-M. Muller et al., *Handbook of Floating-Point Arithmetic*, 2nd ed., Birkhäuser, 2018.